

Sistema di Raccomandazione Ibrido per Hotel

Indice

- 1. Introduzione: di cosa si tratta
- 2. Come funziona, in tre passi (con un esempio guidato)
- 3. Il "gusto" di un hotel: le 5 dimensioni delle preferenze
- 4. Il dataset di prova
- 5. La memoria del sistema: come le preferenze evolvono nel tempo
- 6. Il motore che ottimizza le preferenze
- 7. Come troviamo l'hotel giusto
- 8. Capire il linguaggio naturale: pregi e limiti
- 9. Le metriche: come sappiamo che funziona
- 10. Conclusioni

1. Introduzione: di cosa si tratta

Questo progetto è un sistema di raccomandazione di hotel che funziona come un concierge che impara i gusti di chi gli scrive, conversazione dopo conversazione. Invece di far compilare all'utente dei filtri rigidi (prezzo minimo/massimo, numero di stelle, categoria), il sistema capisce richieste espresse in linguaggio naturale — "cerco un resort di lusso per rilassarmi", "ho un budget ridotto" — e le traduce in una raccomandazione concreta.

Per farlo, combina tre componenti molto diversi tra loro, che si passano il lavoro in sequenza: un modello linguistico che legge e interpreta il messaggio, un algoritmo di intelligenza artificiale che tiene aggiornato un profilo di preferenze e lo trasforma in un punto di ricerca preciso, e un motore di ricerca geometrico che trova l'hotel più vicino a quel punto in un catalogo.

2. Come funziona, in tre passi

Il modo più semplice per capire il sistema è seguire una singola richiesta dall'inizio alla fine. Prendiamo un esempio reale, preso da uno dei test effettuati durante lo sviluppo.

Esempio guidato

Query: *"Sto organizzando un viaggio a Roma. Cerco un'esperienza esclusiva: mi serve un hotel con il massimo del comfort e un'eccellente valutazione. Il prezzo non è un problema, sono disposto a pagare qualsiasi cifra per avere standard altissimi."*

Risposta del sistema: Il sistema propone un hotel a Roma con recensioni molto positive, categoria di lusso e un prezzo elevato, spiegando perché si adatta alla richiesta.

Dietro questa singola risposta, sono successe tre cose in sequenza:

Passo 1 — Capire cosa vuole l'utente

Un modello linguistico (un piccolo LLM eseguito localmente, senza bisogno di internet) legge il messaggio e ne estrae gli aspetti rilevanti: la città (Roma), quanto comfort/luusso viene richiesto (molto alto), quanto l'utente è disposto a spendere (molto, esplicitamente "il prezzo non è un problema"). Il risultato è una struttura ordinata di valori numerici, non più testo libero.

Passo 2 — Aggiornare e ottimizzare il profilo

Questi valori vengono uniti al profilo di preferenze che il sistema ha costruito nelle conversazioni precedenti (se è la prima interazione, si parte da un profilo neutro). Il profilo aggiornato viene poi passato a un algoritmo di intelligenza artificiale — il cuore del sistema, addestrato separatamente — che lo traduce in un punto preciso in uno spazio di ricerca a 5 dimensioni (le 5 dimensioni sono descritte nella Sezione 3).

Passo 3 — Trovare l'hotel più vicino

Il sistema cerca, tra tutti gli hotel disponibili (eventualmente filtrati per città), quello che si trova geometricamente più vicino a quel punto. Trovato l'hotel, il modello linguistico genera una risposta naturale che lo presenta all'utente, motivando la scelta.

3. Il "gusto" di un hotel: le 5 dimensioni delle preferenze

Per poter confrontare in modo automatico ciò che un utente desidera con ciò che un hotel offre, entrambi vengono descritti con lo stesso linguaggio: 5 numeri, ciascuno compreso tra 0 e 1.

Dimensione	Cosa rappresenta	Da dove viene il valore
Reputazione	Quanto sono buone le recensioni	Valutazione media (da 1 a 5 stelle), riportata in scala 0-1
Comfort	Quanto è lussuosa la struttura	Categoria dell'hotel: Budget/Motel, Standard, o Lusso/Resort
Popolarità	Quanto è conosciuto/recensito	Numero di recensioni ricevute, confrontato con gli altri hotel
Leisure	Vacanza/relax oppure lavoro	Il tipo di soggiorno per cui l'hotel è più adatto
Prezzo	Quanto costa per notte	Prezzo per notte, confrontato con gli altri hotel

Tabella 1 — Le 5 dimensioni che descrivono sia gli hotel sia le preferenze di un utente.

Un aspetto importante, verificato più volte durante i test, è che queste 5 dimensioni sono indipendenti tra loro: un utente può volere un hotel di lusso ma a un prezzo contenuto, oppure essere disposto a spendere molto senza cercare necessariamente il massimo comfort. Il sistema tratta ciascuna dimensione separatamente, così da poter rappresentare anche queste combinazioni meno ovvie.

Esempio — preferenze indipendenti

Query: "Cerco un hotel di categoria Lusso/Resort ma con un prezzo onesto, senza spendere una follia."

Risposta del sistema: Il profilo si aggiorna con comfort alto e prezzo medio-basso, in direzioni opposte tra loro — esattamente come richiesto, senza che l'uno "trascini" l'altro.

4. Il dataset di prova

In assenza di un catalogo alberghiero reale, il sistema è stato testato su un dataset sintetico di 500 hotel generati automaticamente, con città, categoria, numero di recensioni, valutazione e prezzo plausibili. Il sistema è progettato per accettare anche un dataset reale con lo stesso formato: le stesse formule di normalizzazione della Tabella 1 verrebbero applicate automaticamente.

Un limite di questo dataset, emerso chiaramente durante la fase di test (approfondito nella Sezione 9), è la sua dimensione ridotta: 500 hotel non sono sempre sufficienti a coprire in modo denso uno spazio di ricerca a 5 dimensioni. Per alcune combinazioni di preferenze molto specifiche, semplicemente non esiste, tra questi 500, un hotel davvero vicino a ciò che viene richiesto.

5. La memoria del sistema: come le preferenze evolvono nel tempo

Ogni nuovo messaggio non sovrascrive il profilo precedente, ma lo aggiorna gradualmente: il nuovo profilo è per l'80% quello che emerge dal messaggio corrente, e per il 20% quello che il sistema sapeva già. In formula:

$$\text{nuovo profilo} = 20\% \times \text{profilo precedente} + 80\% \times \text{nuova richiesta}$$

Questo produce due comportamenti utili nella pratica. Primo: se l'utente non menziona un aspetto in un dato messaggio, quell'aspetto resta esattamente come prima (eredita il valore precedente) — non serve ripetere ogni preferenza ad ogni turno. Secondo: richieste ripetute nella stessa direzione rafforzano il profilo in modo naturale, mentre un cambio di rotta esplicito lo sposta rapidamente nella nuova direzione.

Esempio — eredità e persistenza

Query: "Turno 1: "Ho un budget molto ridotto, cerco il prezzo più basso possibile" (prezzo scende molto). Turno 2: "Cerco un hotel a Roma" (nessuna menzione del prezzo)."

Risposta del sistema: Nel turno 2 il valore di prezzo resta invariato rispetto al turno 1: il sistema non lo "dimentica" solo perché non viene ripetuto.

Questo profilo, insieme alla cronologia della conversazione, viene salvato in modo persistente per ciascun utente: se si torna a parlare con il sistema in una sessione successiva, la traiettoria costruita fino a quel momento viene ricaricata, invece di ripartire da zero.

6. Il motore che ottimizza le preferenze

Una volta calcolato il profilo (i 5 numeri della Sezione 3), il sistema deve trasformarlo in un punto di ricerca affidabile. Questo compito è affidato a un algoritmo di apprendimento per rinforzo multi-agente (in sigla, MADDPG), addestrato separatamente prima di essere collegato al resto del sistema.

L'idea centrale è semplice da descrivere anche senza entrare nei dettagli matematici: durante l'addestramento, il sistema si esercita ripetutamente a riprodurre punti bersaglio generati a caso in uno spazio a 5 dimensioni, ricevendo un piccolo premio ogni volta che si avvicina al bersaglio e nessun premio quando se ne allontana. Ripetendo questo esercizio migliaia di volte, il sistema impara una regola generale per tradurre qualunque combinazione di preferenze in un'azione coerente — un po' come imparare a lanciare freccette allenandosi su bersagli sempre diversi, finché il gesto diventa affidabile indipendentemente da dove si trovi il bersaglio quella volta.

Tecnicamente, il compito è diviso tra 5 reti neurali indipendenti (una per dimensione), che si coordinano tra loro durante l'addestramento pur restando indipendenti quando vengono effettivamente usate. Questa scelta architetturale (nota in letteratura come apprendimento multi-agente) permette a ciascuna dimensione di specializzarsi, mantenendo comunque una coerenza complessiva tra le 5.

7. Come troviamo l'hotel giusto

Una volta ottenuto il punto di ricerca dal motore RL, trovare l'hotel corrispondente è concettualmente semplice: si cerca, tra gli hotel della città richiesta (o di tutte le città, se non ne è stata specificata una), quello la cui descrizione a 5 numeri è più vicina al punto cercato — un confronto puramente geometrico (distanza euclidea), simile a cercare il punto più vicino su una mappa.

Un affinamento importante, introdotto durante i test, riguarda le dimensioni che l'utente non ha mai menzionato esplicitamente. Se restano al valore neutro di default, non è corretto trattarle alla pari di quelle su cui l'utente si è espresso chiaramente: un hotel non dovrebbe essere scartato solo perché ha una popolarità "casuale" su un aspetto a cui, di fatto, all'utente non interessa nulla. Per questo, le dimensioni mai discusse pesano meno nel calcolo della vicinanza, mentre quelle esplicitamente richieste pesano per intero — e questo peso resta valido anche nei turni successivi in cui non vengono ripetute, dato che il valore stesso continua a essere ereditato dalla memoria del sistema (Sezione 5).

8. Capire il linguaggio naturale

Il modello linguistico usato per leggere le richieste (chiamato qwen2.5:3b) è volutamente piccolo e leggero, per poter girare localmente senza bisogno di server esterni o costi di utilizzo. Questa scelta ha un costo: un modello di queste dimensioni comprende bene le richieste dirette, ma può confondersi su formulazioni indirette o insolite. Durante i test sono emersi alcuni casi concreti, ciascuno poi corretto.

Un problema di direzione

Prima della correzione

Query: *"Devo andare a Napoli e ho un budget molto ridotto. Il prezzo deve essere il più basso possibile."*

Risposta del sistema: Il sistema aveva interpretato questa richiesta come un budget elevato, proponendo un hotel costoso — l'esatto opposto di quanto chiesto.

Causa: il modello confondeva "quanto conta il prezzo per l'utente" con "quanto l'utente vuole spendere", due concetti diversi.

La correzione ha avuto due parti. Prima, il testo che istruisce il modello è stato riscritto con esempi molto più espliciti nelle due direzioni (economico vs costoso). Poi, dato che nessun modello di queste dimensioni è affidabile al 100% su questo tipo di sfumature, è stata aggiunta una rete di sicurezza: un controllo aggiuntivo, indipendente dal modello linguistico, che riconosce esplicitamente espressioni legate al denaro ("budget ridotto", "non bado a spese", "soldi non mancano...") e corregge la direzione se necessario. Dopo la correzione, la stessa identica richiesta produce un prezzo coerentemente basso.

Altri due problemi minori, identificati e corretti

- Invenzione di preferenze mai espresse: in alcuni casi il modello attribuiva un valore a un aspetto mai menzionato dall'utente, invece di lasciarlo invariato. Corretto rafforzando le istruzioni con un esempio esplicito di cosa NON fare.
- Toni dubitativi ingiustificati: a volte la risposta finale conteneva frasi come "mi dispiace, non troviamo nulla di adatto", nonostante il sistema avesse comunque trovato un hotel valido (per costruzione, questo accade sempre). Corretto vietando esplicitamente questo tipo di linguaggio nelle istruzioni di risposta.

9. Le metriche: come sappiamo che funziona

Per verificare la qualità del sistema in modo ripetibile (non solo con conversazioni di prova lette a mano), sono state misurate due grandezze distinte, con un'analogia utile a tenerle separate: quella del tiro a segno.

Concetto	Cosa significa qui
Accuratezza	Quanto, in media, il sistema si avvicina davvero a ciò che viene richiesto
Precisione	Quanto le risposte sono coerenti e ripetibili, indipendentemente da quanto siano vicine al bersaglio

Tabella 2 — Le due grandezze misurate, e cosa rappresentano concretamente.

Queste due grandezze sono state misurate in due condizioni diverse, per capire dove nasce l'eventuale errore: usando solo il motore RL (Sezione 6) isolato dal resto, e usando il sistema completo (RL + ricerca nel dataset reale di 500 hotel).

Condizione	Accuratezza	Precisione
Solo motore RL (isolato)	73%	72%
Sistema completo (+ ricerca hotel)	54%	55%

Tabella 3 — Confronto tra il motore RL isolato e il sistema completo, su un problema a 3 classi (basso/medio/alto).

Un chiarimento sul punto di riferimento: su un problema a 3 classi, indovinare a caso darebbe circa il 33% — il 73% del motore RL isolato indica quindi un apprendimento reale e solido. Il calo nel sistema completo, invece, non indica un errore nella logica del sistema, ma una causa specifica e verificabile: con solo 500 hotel disponibili, non sempre esiste una struttura realmente vicina al profilo richiesto. Una verifica dedicata lo conferma: quando il sistema non sceglie l'hotel migliore in assoluto tra quelli disponibili, la scelta effettiva resta comunque quasi equivalente a quella ideale nella grande maggioranza dei casi — il problema è che, a volte, anche la scelta "ideale" nel dataset non è poi così vicina a quanto richiesto.

Un'osservazione ulteriore, utile a chi volesse approfondire: alcune caratteristiche soffrono più di altre questo effetto. La categoria di un hotel (economico / standard / lusso), ad esempio, assume per natura solo tre valori ben distinti e si presta quasi perfettamente allo schema di misurazione usato — il calo di prestazioni su questa dimensione specifica è infatti il più contenuto tra tutti. Altre dimensioni, come la popolarità, sono invece distribuite in modo molto più squilibrato nel dataset sintetico (la stragrande maggioranza degli hotel ha popolarità bassa), rendendo strutturalmente più difficile soddisfare richieste di hotel molto popolari — un limite del dataset di prova, non della logica di ricerca.

10. Conclusioni

Il sistema realizza con successo l'integrazione tra comprensione del linguaggio naturale, apprendimento automatico e ricerca geometrica in un'unica esperienza conversazionale coerente. Il percorso di test, descritto in questa relazione anche attraverso esempi reali, ha permesso di individuare e correggere diversi problemi concreti e di distinguere chiaramente ciò che è un limite del codice da ciò che è un limite dei dati di prova disponibili — una distinzione che si è rivelata più utile di un singolo giudizio complessivo di "qualità", perché indica con precisione dove investire lo sforzo successivo.